Object-Oriented Design Interview

Info coming soon

Lecture on Object Oriented Design Interview

Pre-OOD Interview

Design a Parking Lot

Design a Online Ticket Booking System

Design a Uber-like Ride Sharing System

Design a Ride Sharing System

Design an Elevator System

Design a Distance Sorting System

Design a TV in The Taxi

Design a Blackjack Game

Design an OS System

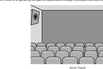
Design an Electronic Management System

05

Design a Movie Ticket Booking System

In this chapter, we'll walk through the design of a Movie Ticket Booking System. It's a common question your ability to model real-world systems and apply object oriented principles to a concrete problem. The goal is to quickly identify the classes that represent key entities in a movie booking system, such as rooms, screenings, and reviews. We'll aim to build a clean and functional structure that supports the core booking flow while keeping the system extensible and readable.

Let's focus on gathering the specific requirements through a simulated interview session.



Requirements Gathering

Here's an example of a typical prompt an interviewer might give:

"Imagine you're trying to build the ticketing system for a movie theater chain or a music venue. You may offer the booking system, browse through the available showtimes, select your preferred seats, and proceed to book them. Within ceremonies, your books are reserved for a limited time before being released back into the pool. The system should also support features like user reviews and ratings for movies, and manage various ticket types and pricing tiers. Let's design a basic version of this ticketing system that handles all of this operation."

To refine the system's scope, the candidate might ask the following questions:

Requirements clarification

Here is an example of how a conversation between a candidate and an interviewer might unfold:

Candidate: Does the system support booking and holding tickets across different cinemas and venues?
Interviewer: Yes, the system should be a centralized hub for multiple cinema chains, with unique locations and rooms.

Candidate: Should we handle different ticket types, such as standard, student, and senior tickets, with different pricing?
Interviewer: Yes, that's a crucial feature. The system should be able to handle different ticket types and apply the correct pricing rules based on the selected ticket type.

Candidate: Does the system support different pricing tiers for seats within the same screening?
Interviewer: Yes, we want to be able to price seats differently, such as normal, premium, or VIP, all offering the ticket price.

Candidate: Can a user book multiple tickets in a single order, and how does the system calculate the total cost?
Interviewer: Users should be able to book multiple tickets for the same screening. The system must calculate the total cost by summing the prices of all selected tickets and applying any relevant discounts.

Candidate: Does the system need to handle payment processing or just reserve tickets?
Interviewer: For this design, let's focus on the core reservation logic. We can assume a separate, third-party system handles the payment processing.

Candidate: What happens when a seat books a ticket for a specific user?
Interviewer: The system should mark the seat as "booked" and associate it with the user's profile and their specific ticket.

Now that we know the core objects and their roles, the next step is to create classes and methods to build the movie ticket booking system.

Movie

The Movie class represents essential details about a specific movie in the system. It focuses on core information, like genre, and duration, data that remains constant across all screenings of that film. It acts as a simple data model from a Screening, which has a movie in a room and show time for a particular showing.

Below is the UML diagram for the Movie class:

Requirements

The Movie class acts as the foundation for the entire system, independent of cinema specifics or scheduling details. This isolation allows the core data to be reused across multiple cinemas and screenings without data duplication.

Seat

The Seat class models individual seats within a specific room. It's a simple data model that stores the seat's unique identifier, its row, and its number. This allows the system to be reused across multiple cinemas and screenings without data duplication.

Alternate approach: We could embed pricing logic directly in the Seat, but this violates Separation of Concerns principle. The design patterns, while more complex, supports future extensions.

Why this design? The Seat class is perfect for a system's seating grid. Focusing solely on seat-specific data like its number and accessibility, it's lightweight and reusable.

While To learn more about the Strategy pattern and its everyday use cases, refer to the **Factory** chapter of this book.

Layout

The Layout class acts as a bridge between individual seats and cinema rooms. It organizes seats into a grid structure defined by rows and columns. It uses a nested map (`Map<Integer, Map<Integer, Seat>>`) for efficient seat lookup, where the outer map links row numbers to inner maps, and the inner map links column number with that row, and the value is the Seat object at that coordinate in the Layout. It uses

It also maintains an index (`Map<String, Seat>`) to locate seats by their seat numbers quickly. This design simplifies managing seats within cinema rooms layouts, ensuring easy access to seat data for booking purposes.

The UML diagram below illustrates this structure:

Layout
- int rows
- int columns
- Map<String, Seat> seatByNumber
- Map<Integer, Map<Integer, Seat>>[]seatMap
+ Layout(int, int, int)
+ addSeat(int, int, int,

```
    private final Room room;
    private final LocalDateTime startTime;
    private final LocalDateTime endTime;

    public ScreeningMovie(Movie movie, Room room, LocalDateTime startTime, LocalDateTime endTime) {
        this.movie = movie;
        this.room = room;
        this.startTime = startTime;
        this.endTime = endTime;
    }

    public Duration getDuration() {
        return Duration.between(startTime, endTime);
    // of getter and setter methods are omitted for brevity
}
```

The class provides a getDuration() method to calculate the duration of the screening based on its start and end times.

Ticket

The Ticket class represents a single unit of purchase in the ticket booking system. It connects a Screening (scheduled movie instance) with a specific Seat and assigns a price at the time of booking. This design ensures that the ticket encapsulates all necessary details for a single booking, making it central to the ticketing process.

```
public class Ticket {
    private final Screening screening;
    private final Seat seat;
    private final BigDecimal price;

    public Ticket(Screening screening, Seat seat, BigDecimal price) {
        this.screening = screening;
        this.seat = seat;
        this.price = price;
    }

    // getter and setter methods are omitted for brevity
}
```

Order

The Order class represents a single transaction in the movie ticket booking system. It encapsulates a collection of Ticket objects purchased together and records the date and time of the order (orderDate). This design ensures that all tickets associated with a single booking are grouped and tracked cohesively.

```
public class Order {
    private List<Ticket> tickets;
    private final LocalDateTime orderDate;

    public Order(LocalDateTime orderDate) {
        this.tickets = new ArrayList<>();
        this.orderDate = orderDate;
    }

    public void addTicket(Ticket ticket) {
        tickets.add(ticket);
    }

    // Calculates the total price of all tickets in the order
    public BigDecimal calculateTotalPrice() {
        return tickets.stream().map(Ticket::getPrice).reduce(BigDecimal.ZERO, BigDecimal::add);
    }

    // getter and setter methods are omitted for brevity
}
```

ScreeningManager

The ScreeningManager class serves as a centralized manager for showtimes and tickets within the movie ticket booking system. It maintains mappings between Movie objects and their corresponding Screening instances, as well as between Screening objects and their associated Ticket objects. This design ensures that the system can dynamically manage showtimes and ticket bookings while maintaining clear relationships between components.

Here is the implementation of this class.

```
// Manages the relationships between movies, screenings, and tickets in the booking system
public class ScreeningManager {
    // Maps movies to their scheduled screenings
    private final Map<Movie, List<Screening>> screeningsByMovie;
    // Maps screenings to tickets sold for that screening
    private final Map<Screening, List<Ticket>> ticketsByScreening;

    public ScreeningManager() {
        this.screeningsByMovie = new HashMap<>();
        this.ticketsByScreening = new HashMap<>();
    }

    public void addScreening(Movie movie, Screening screening) {
        screeningsByMovie.computeIfAbsent(movie, k -> new ArrayList<>()).add(screening);
    }

    // Returns all screenings for a specific movie
    public List<Screening> getScreeningsForMovie(Movie movie) {
        return screeningsByMovie.getOrDefault(movie, new ArrayList<>());
    }

    public void addTicket(Screening screening, Ticket ticket) {
        ticketsByScreening.computeIfAbsent(screening, k -> new ArrayList<>().add(ticket);
    }

    // Returns all tickets sold for a specific screening
    public List<Ticket> getTicketsForScreening(Screening screening) {
        return ticketsByScreening.getOrDefault(screening, new ArrayList<>());
    }

    // Calculates which seats are still available for a screening
    public List<Seat> getAvailableSeats(Screening screening) {
        List<Seat> allSeats = screening.getRoom().getSeats();
        List<Ticket> bookedTickets = getTicketsForScreening(screening);

        List<Seat> availableSeats = new ArrayList<>(allSeats);
        for (Ticket ticket : bookedTickets) {
            availableSeats.remove(ticket.getSeat());
        }
        return availableSeats;
    }
}
```

- The addScreening and addTicket methods use computeIfAbsent to dynamically initialize the lists for a movie's screenings or a screening's tickets if they do not already exist.
- The getAvailableSeats method determines which seats are available for a given screening. It retrieves all seats from the room's Layout and removes seats that are already associated with tickets for the screening. This logic ensures accurate seat availability by dynamically reflecting the current booking status.

Implementation choice: We used a Map<Movie, List<Screening>> for movie-to-screenings mappings and a Map<Screening, List<Ticket>> for screenings-to-ticket mappings. These maps provide O(1) lookup, optimizing search and booking operations. An alternative could be a List<Screening> and List<Ticket> with manual filtering to find relevant entries, but this would result in O(n) search times. The map-based approach is more efficient and scalable for managing large numbers of screenings and tickets.

MovieBookingSystem

The MovieBookingSystem class serves as a facade for the movie ticket booking system, providing a simplified interface for managing the core data and operations. It abstracts away the complexities of interacting with underlying components, such as ScreeningManager, while maintaining seamless integration with Movie, Cinema, Screening, and Seat.

Below is the code implementation of this class:

```
// Manages the complete movie booking system operations
public class MovieBookingSystem {

    private final List<Movie> movies;
    private final List<Cinema> cinemas;
    private final ScreeningManager screeningManager;

    public MovieBookingSystem() {
        this.movies = new ArrayList<>();
        this.cinemas = new ArrayList<>();
        this.screeningManager = new ScreeningManager();
    }

    public void addMovie(Movie movie) {
        movies.add(movie);
    }

    public void addCinema(Cinema cinema) {
        cinemas.add(cinema);
    }

    public void addScreening(Movie movie, Screening screening) {
        screeningManager.addScreening(movie, screening);
    }

    // Books a ticket for a specific seat at a screening
    public void bookTicket(Screening screening, Seat seat) {
        BigDecimal price = seat.getPricingStrategy().getPrice();
        Ticket ticket = new Ticket(screening, seat, price);
        screeningManager.addTicket(screening, ticket);
    }

    // Returns all screenings for a specific movie
    public List<Screening> getScreeningsForMovie(Movie movie) {
        return screeningManager.getScreeningsForMovie(movie);
    }

    // Returns all available seats for a screening
    public List<Seat> getAvailableSeats(Screening screening) {
        return screeningManager.getAvailableSeats(screening);
    }

    // Returns the number of tickets sold for a screening
    public int getTicketCount(Screening screening) {
        return screeningManager.getTicketsForScreening(screening).size();
    }

    // Returns the list of tickets for a screening
    public List<Ticket> getTicketsForScreening(Screening screening) {
        return screeningManager.getTicketsForScreening(screening);
    // of getter and setter methods are omitted for brevity
}
```

Key functionalities include adding movies and cinemas, retrieving available seats for a specific screening, finding screenings for a given movie, and booking tickets.

The bookTicket method dynamically calculates the price of a ticket using the associated seat's PricingStrategy. It then creates a new Ticket instance, associating it with the specified screening and seat, and stores it in the ScreeningManager.

Deep Dive Topics

Now that the basic design is complete, the interviewer might ask you to enhance the movie ticket system's functionality or accommodate more complex use cases.

Handling concurrent bookings

In OOD interviews, concurrency is often discussed for systems like movie ticket booking, where multiple users interact simultaneously. Concurrency adds complexity and requires understanding basic threading concepts. During an interview, ask the interviewer if concurrency needs to be handled in the system.

Problem: Race condition

A race condition occurs when two users attempt to book the same seat at the same time, potentially leading to both receiving confirmation, causing a double-booking. This undermines the system's reliability and user experience.

Solution: Pessimistic and Optimistic locking

To prevent race conditions, we can use locking mechanisms to ensure only one user books a seat at a time, turning parallel actions into sequential ones. There are two common approaches: pessimistic locking and optimistic locking, each with distinct trade-offs.

Pessimistic Locking: This strategy acquires an exclusive lock on a seat at the start of a user's booking process, preventing concurrent access by other users until the lock is released. The lock is held throughout the transaction, spanning seat selection, booking confirmation, or payment, ensuring only one user gains access to the seat. For instance, when a user selects a seat, the system locks it until the booking is finalized or abandoned. To mitigate the risk of indefinite locks due to system failures, a timeout mechanism (e.g., 30 seconds) automatically releases the lock if the transaction remains incomplete. Pessimistic locking is well-suited for scenarios with high contention, where multiple users frequently attempt to book the same seat, as it guarantees exclusive access. Its drawback is increased latency due to lock acquisition and release overhead, which can impact system performance under heavy load.

Optimistic Locking: This strategy avoids locking seats during the booking process, instead verifying seat availability at the transaction's final stage. If another user has booked the seat in the interim, the transaction fails, requiring the user to retry. Optimistic locking is lightweight and efficient in low-contention scenarios, where simultaneous booking attempts for the same seat are rare, minimizing the need for retries. However, in high-contention environments, frequent conflicts can lead to multiple retries, degrading user experience, and increasing system load. This approach relies on atomic checks (e.g., using synchronized blocks or database transactions) to ensure consistency during the final validation.

We provide implementation for both locking strategies below.

Implementation: Pessimistic locking

To implement pessimistic locking, we introduce a SeatLockManager class that uses Java's concurrency features to manage temporary seat locks. It uses a ConcurrentHashMap for thread-safe access and synchronizes critical methods to prevent race conditions during lock creation and cleanup. Each lock has a timeout to prevent permanent holds if a process fails.

```
public class SeatLockManager {
    private final Map<String, SeatLock> lockedSeats = new ConcurrentHashMap<>();
    private final Duration lockDuration;

    public SeatLockManager(Duration lockDuration) {
        this.lockDuration = lockDuration;
    }

    public synchronized boolean lockSeat(Screening screening, Seat seat, String userId) {
        String lockKey = generateLockKey(screening, seat);

        // Clean up lock if expired (on-demand cleanup when another process attempts to lock)
        cleanupLockIfExpired(lockKey);
        // Check if a seat is already locked
        if (isLocked(screening, seat)) {
            return false;
        }

        // Create a new lock with expiration time
        SeatLock lock = new SeatLock(userId, LocalDateTime.now().plus(lockDuration));
        lockedSeats.put(lockKey, lock);
        return true;
    }

    public synchronized boolean isLocked(Screening screening, Seat seat) {
        String lockKey = generateLockKey(screening, seat);

        // Clean up lock if expired (on-demand cleanup)
        cleanupLockIfExpired(lockKey);

        // If we reach here, either no lock exists or it's valid
        return lockedSeats.containsKey(lockKey);
    }

    private void cleanupLockIfExpired(String lockKey) {
        SeatLock lock = lockedSeats.get(lockKey);
        if (lock != null && lock.isExpired()) {
            lockedSeats.remove(lockKey);
        }
    }

    private String generateLockKey(Screening screening, Seat seat) {
        return screening.getId() + "-" + seat.getSeatNumber();
    }

    // SeatLock inner class
    private static class SeatLock {
        private final String userId;
        private final LocalDateTime expirationTime;

        public SeatLock(String userId, LocalDateTime expirationTime) {
            this.userId = userId;
            this.expirationTime = expirationTime;
        }

        public boolean isExpired() {
            return LocalDateTime.now().isAfter(expirationTime);
        }

        public String getUserId() {
            return userId;
        }
    }
}
```

- **lockSeat(Screening, Seat, String)**: Synchronously attempts to lock a seat for a user by generating a unique lock key, cleaning up any expired locks, and checking if the seat is currently locked. If the seat is available, it creates and stores a new lock with an expiration timestamp to ensure the lock is temporary.
- **isLocked(Screening, Seat)**: Checks if a valid lock exists after cleaning up expired locks.
- **cleanupLockIfExpired(String)**: Removes expired locks from lockedSeats to free the seat.

Implementation: Optimistic locking

In the optimistic locking approach, the ScreeningManager class is updated to verify seat availability just before finalizing a booking, without acquiring a persistent lock beforehand. The synchronized keyword is used to ensure that the availability check and the booking operation execute atomically, preventing race conditions that could lead to double-booking.

```
// Simplified optimistic locking in ScreeningManager
public synchronized Ticket bookSeatOptimistically(Screening screening, Seat seat) {
    // First check if a seat is available (optimistic)
    if (!isSeatBooked(screening, seat)) {
        throw new IllegalStateException("Seat is already booked");
    }

    // Create ticket - at this point, we're optimistically assuming
    // the seat is still available
    BigDecimal price = seat.getPricingStrategy().getPrice();
    Ticket ticket = new Ticket(screening, seat, price);

    // Add to booking system - this effectively "reserves" the seat
    ticketsByScreening.computeIfAbsent(screening, k -> new ArrayList<>().add(ticket);

    return ticket;
}

// Helper method to check if a seat is already booked
private boolean isSeatBooked(Screening screening, Seat seat) {
    List<Ticket> tickets = getTicketsForScreening(screening);
    return tickets.stream().anyMatch(ticket -> ticket.getSeat().equals(seat));
}
```

bookSeatOptimistically(Screening, Seat): Checks if the seat is booked, creates a Ticket, and atomically adds it to the screening's ticket list.

This implementation is lightweight and suitable when race conditions are infrequent, but it may require users to retry if a booking fails due to a conflict.

> **Interview Tip**: In an OOD interview, discussing concurrency shows your ability to handle advanced scenarios. For the movie ticket booking system, pessimistic locking is often preferred for popular screenings with high contention, while optimistic locking suits simpler cases with fewer conflicts. Ask the interviewer which approach aligns with their expectations, and implement a basic solution like those above.

Wrap Up

In this chapter, we gathered requirements for the Movie Ticket Booking System through a series of thoughtful questions and answers. We then identified the core objects involved, designed the class structure, and implemented the key components of the system.

A key takeaway from this design is the significance of modularity and adherence to the single responsibility principle. Each component, such as the Movie, ScreeningManager, Seat, and Order classes, handles a distinct responsibility, ensuring the system remains maintainable and adaptable for future enhancements.

Our design choices, such as separating Movie and Screening or using a strategy pattern for pricing, prioritize flexibility and scalability. An alternative, like merging Screening and Ticket, might simplify the model but could complicate individual seat management. In an interview, revisiting these decisions and explaining their rationale showcases your critical thinking ability.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
help@bytebytego.com
Report a Bug

Resources
YouTube
Blog
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Leo

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.